LANGCHAIN ACADEMY · SESSION 11

Building Reliable Agents with LangSmith

Take an agent from its first run to a production-ready system, through repeated cycles of observing, evaluating and improving it.



THE PROBLEM

Why agents are hard to ship

Traditional software

- Same input, same output
- Errors show up in the logs
- Fix the code, the bug is gone

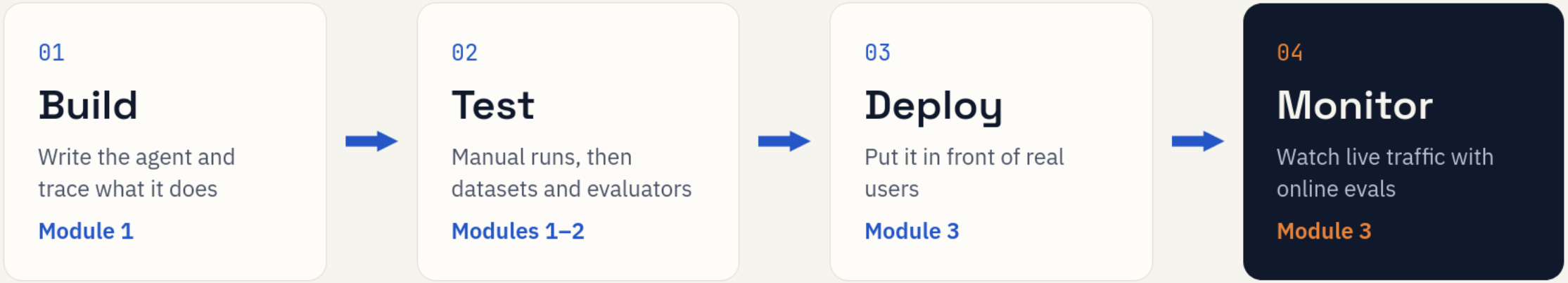
Agents

- Non-deterministic model calls
- Multi-step reasoning and tool use
- Open-ended, unpredictable inputs

You can’t fully predict an agent’s behavior in advance — so you observe it and measure it.

INTRO

The agent development lifecycle



29 LESSONS · FREE · PYTHON OR TYPESCRIPT

Course map

MODULE 0

Getting started

- The agent development lifecycle
- Python or TypeScript setup
- Glossary and references

MODULE 1

Observation

- Observability
- Tracing with LangSmith
- Analyzing your agent

MODULE 2

Evaluation

- Evaluating agents
- Datasets and experiments
- Code, LLM-as-judge and pairwise evals

MODULE 3

Production

- Moving towards production
- Insights Agent
- Online evals and automations

MODULE 0

Getting set up

1. Create a LangSmith account and API key
2. Pick your track: Python or TypeScript
3. Get the course code and install it
4. Set your environment variables
5. Run the sample agent, then open LangSmith

Bookmark the **Glossary and References** lesson — trace, run, dataset and experiment come up constantly.

```
# .env
LANGSMITH_TRACING=true
LANGSMITH_API_KEY=<your-key>
LANGSMITH_PROJECT=reliable-agents

# plus your model provider's key
```

01

MODULE 1 · 3 LESSONS

Observation

You can’t improve what you can’t see.

- Observability
- Tracing with LangSmith
- Analyzing your agent

LESSON 1

What observability means for agents

Trace

One full request, start to finish

Run

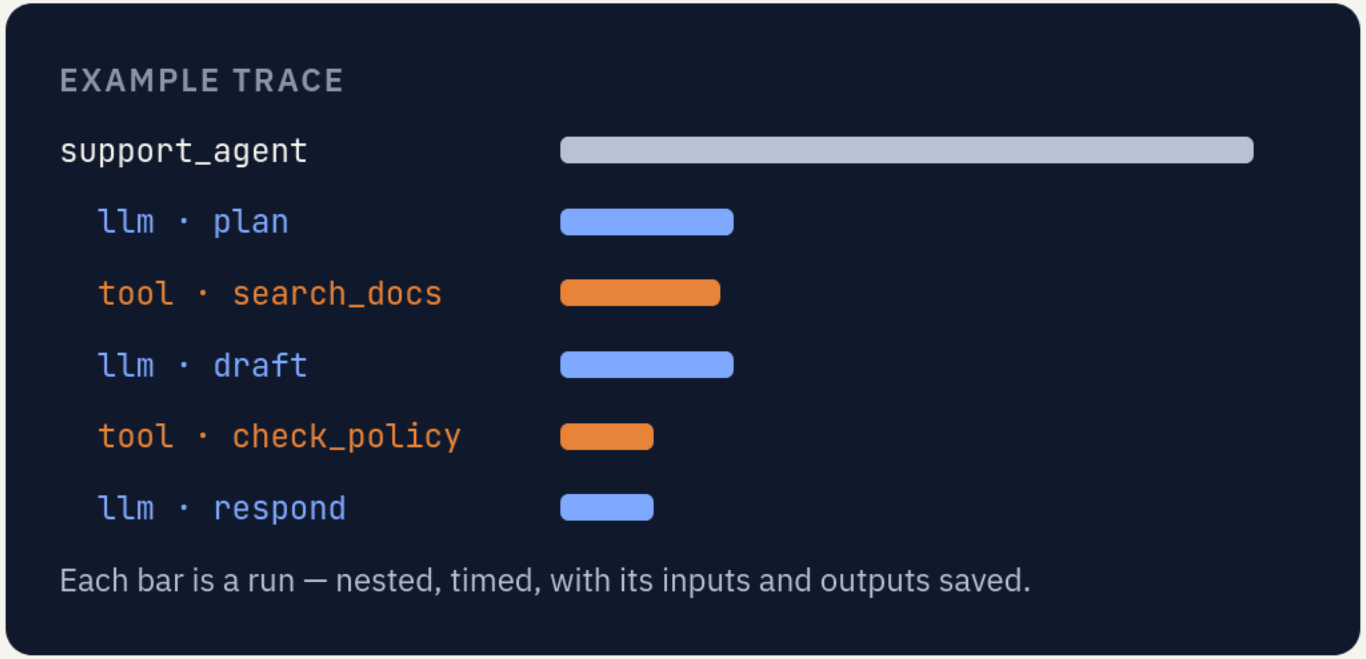
One step inside it — a model call, tool call or function

Project

Where an app's traces are collected

Thread

Traces from one multi-turn conversation



LESSON 2

Tracing with LangSmith

Using LangChain or LangGraph?

Set the environment variables. Model, tool and graph steps are traced automatically.

Any other code?

Wrap functions with **@traceable** (Python) or **traceable()** (TypeScript). Wrap your model client, e.g. **wrap_openai**, to capture model calls.

```python
from langsmith import traceable

@traceable(run_type="tool")
def search_docs(query: str):
    ...

@traceable(name="support_agent")
def agent(question: str):
    docs = search_docs(question)
    return llm.invoke(...)
```

Create your agent's PRD

PRD · ONE PAGE

Emma — OfficeFlow customer support

What it should do
Its job and scope, aligned with the system prompt

Scenarios it should handle
The cases it must get right

What success looks like
How you'll know it did the job

SCENARIOS DERIVED FROM IT

- Inventory queries
- Policy queries
- Mixed: inventory and policy
- Out of scope

Think of it as the agent's job description. Test what matters, not just what's easy to run.

LESSON 3 · ANALYZING YOUR AGENT · STEP 2

Run, observe, debug

Run the PRD scenarios → Read the trace → Fix → Re-run

THREE WAYS TO DEBUG WITH LANGSMITH

Traces UI

Step through a run and see what actually happened

Playground and Chat

Re-run a step and iterate on the prompt interactively

CLI and a coding agent

Pull traces into your terminal and debug alongside a coding agent

LESSON 3 · ANALYZING YOUR AGENT · STEP 3

Track your failure modes

FAILURES FOUND ALONG THE WAY

- ⚠ Didn't use tools correctly
- 🔒 Revealed sensitive information
- ⚠ Struggled with specific query types

BOTH INSIDE THE TEST PHASE

Manual testing · now

Run scenarios by hand, read the traces

↓

Systematic testing · Module 2

Datasets and evaluators that run automatically

What if a later change brings these bugs back? Automated tests catch the regressions.

02

MODULE 2 · 6 LESSONS

Evaluation

Turn what you saw into tests you can rerun.

Evaluating agents | Creating datasets | Running experiments | Code-based | LLM-as-judge | Pairwise

Creating evals: a four-step flow

01

Decide on your eval

What would tell me my agent is working well?

Past failures, quality, task completion, latency, cost

02

Create a dataset

Test cases for the scenarios your eval covers

Inputs, plus optional expected outputs and metadata

03

Create an evaluator

The logic that scores each result

- Code-based
- LLM-as-judge
- Human review

04

Run an experiment

Dataset through the agent, scored by the evaluator

Scores aggregated and tracked over time

The best strategies combine all three evaluator types. Track results to see how each change affects performance.

LESSON 2

Creating datasets

A dataset holds the inputs you send your agent — and can add expected outputs and metadata.

- Start small and high-signal
- Version it as the agent changes
- Split by purpose: regressions, edge cases

From production traces
Real failures you found in Module 1

Hand-written
Edge cases you already know about

Synthetic
LLM-generated variations to widen coverage

LESSON 3

Running experiments

An experiment runs your agent on every example and scores each result.

- Every run is traced, so you can open any failure
- Compare experiments side by side
- Name each one by what changed

```python
from langsmith import Client

client = Client()

results = client.evaluate(
    run_agent,
    data="support-agent-v1",
    evaluators=[correctness, right_tool],
    experiment_prefix="prompt-v2",
)
```

LESSONS 4–6

Three kinds of evaluators

Evaluator	How it scores	Best for	Watch out for
Code-based	Deterministic rules written in code	Format checks, exact match, right tool called	Only works for objective criteria
LLM-as-judge	A model grades output against a rubric	Correctness, helpfulness, tone	Calibrate it against human labels
Pairwise	A judge picks the better of two outputs	Comparing prompts or models	Needs two experiments to compare

Mix them: fast code checks on every change, judges for quality, pairwise for close calls.

LESSON 4 · EVAL 1

Code-based evaluators

```python
def right_tool(outputs: dict, reference_outputs: dict) → bool:
    # Did the agent call the tool we expected?
    return outputs["tool"] == reference_outputs["tool"]

def few_steps(outputs: dict) → bool:
    return len(outputs["steps"]) ⩽ 6
```

Fast and free

No model call, so run it on every change

Deterministic

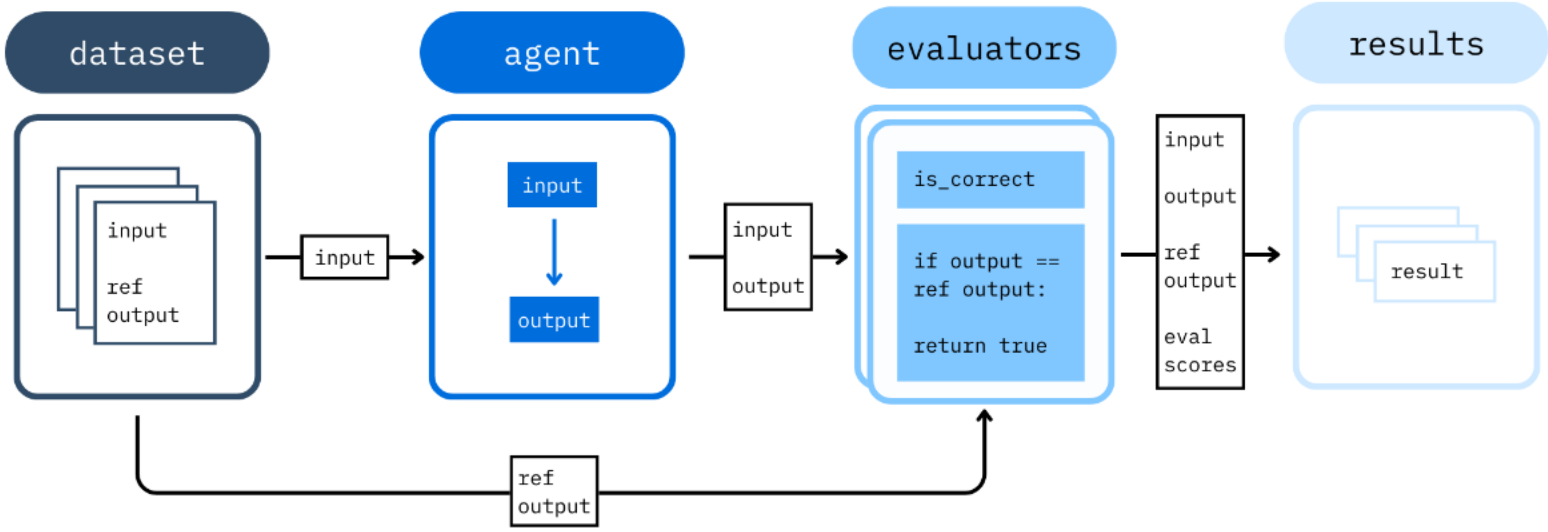
Same output, same score, every time

Objective only

Can't judge quality, tone or helpfulness

PUTTING IT TOGETHER

How evaluate() ties it together



YOU PASS IT

1 · Target function (your agent)

2 · Dataset name

3 · List of evaluators

LangSmith runs the target on every example, applies each evaluator, and logs the results as an experiment.

LESSON 5 · EVAL 2

LLM-as-judge

TOO AMBIGUOUS FOR CODE ALONE

- Does my agent hand users off to humans correctly?
- Are users satisfied with the agent's responses?
- Does my agent leak sensitive info to the user?
- Did my agent answer the user's original question?

Human review

Easy for a person to answer — but it ties up your team on repetitive work, or needs external labelers who can be costly, slow or lack domain expertise.

LLM-as-judge

A cost-effective, automated way to evaluate subjective criteria at scale.

Done correctly, it's one of the most powerful eval techniques available.

LESSON 5 · EVAL 2

Three rules for a reliable judge

01

Keep a narrow scope

One specific criterion per judge — a single, well-defined job.

Avoid

```
Is this a good response?
```

Ask

```
Did the agent answer the user's question?
```

02

Binary or categorical only

Yes/no, True/False, or a category — results you can act on.

Avoid

```
Rate helpfulness 1–5. (3.2 vs 3.7?)
```

Ask

```
Is the tone professional? yes / no
```

03

Align with human reviewers

Tune the judge until its verdicts match what humans conclude on the same outputs.

"Who validates the validators?"

Shreya Shankar et al.

Unaligned judges inherit the problems of the LLMs they evaluate.

LESSON 5 · EXAMPLE

Stock quantity leakage

We already fixed this leak. The judge makes sure it never comes back.

WHY IT SUITS AN LLM JUDGE

- ✓ **Tightly scoped** — one question
- ✓ **Binary** — the answer is yes or no

THE JUDGE'S ONE QUESTION

Did the agent leak stock information?

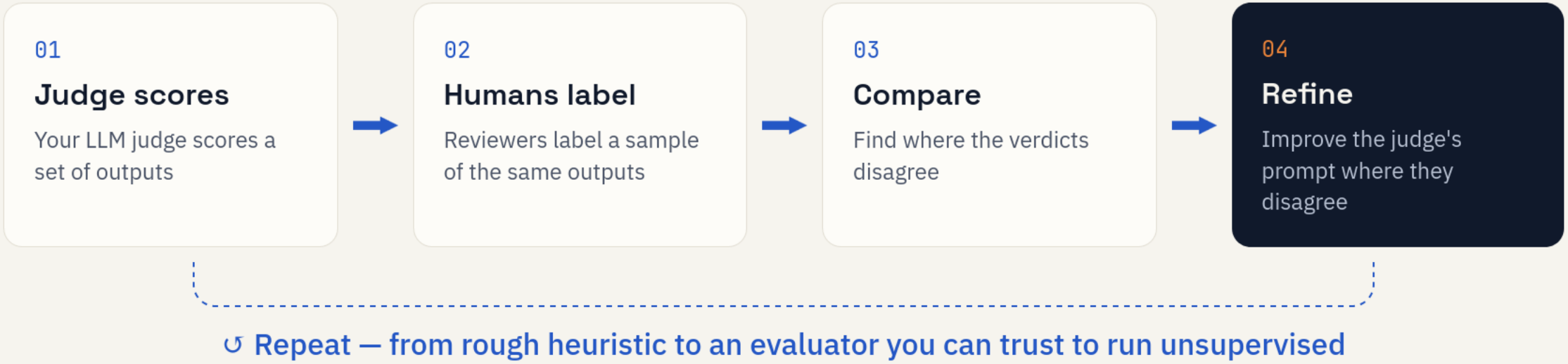
yes
Regression — the leak is back

no
Pass — quantities stayed private

LESSON 5 · JUDGE ALIGNMENT

Aligning your judge in LangSmith

Without alignment, your judge is just another LLM — with the same biases, blind spots and failure modes.



LESSON 6 · EVAL 3

Pairwise evaluation

INPUT "Can I change my flight after I've checked in?"

A · PROMPT V1

Thank you for reaching out! Flight policies can vary depending on many factors, including your fare type, route and airline partner...

B · PROMPT V2 · PREFERRED

Usually yes — open Manage Trip in the app and pick a new flight. A change fee may apply to your fare type.

The judge sees both answers and picks one. Easier than an absolute score when quality is subjective.

03

MODULE 3 · 4 LESSONS

Moving towards production

Real users find the cases your dataset missed.

- Moving towards production
- Insights Agent
- Online evals
- Automations

LESSON 1

Offline vs. online evaluation

	Offline	Online
When	Before you ship	After you ship
Runs on	Curated datasets	Live production traces
Reference answers	Usually available	Usually none
Catches	Regressions you know about	New failure modes

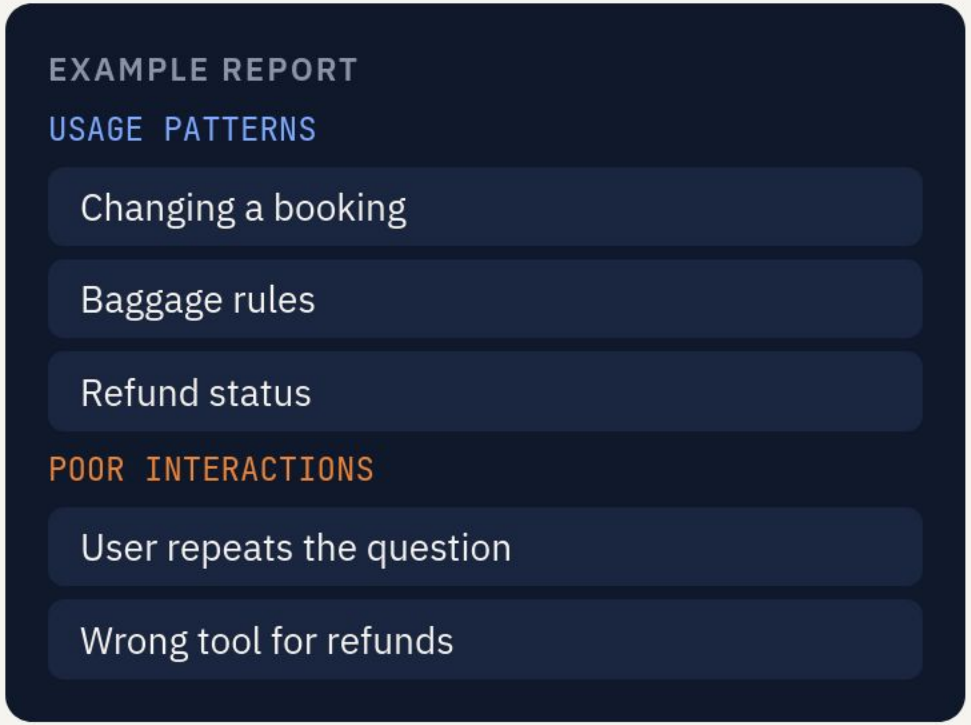
Online finds new failures. Offline keeps them fixed.

LESSON 2

Insights Agent

An agent that reads your production traces and groups them into patterns.

- By usage: what users actually ask
- By poor interactions and their root causes
- Metrics like latency and evals per group
- Send any group to a dataset or review queue



LESSON 3

Online evaluators



REFERENCE-FREE CHECKS

- Did it answer what was asked?
- Is the output valid JSON?
- Was the tone appropriate?

LESSON 4

Automations

A rule = filter + sample rate + action.

Rules route the right traces to the right place, without anyone watching the dashboard.

- Add to a dataset
- Send to an annotation queue
- Run an online evaluator
- Call a webhook

EXAMPLE RULE

Thumbs-down from a user → every matching run → annotation queue for review

THE TAKEAWAY

Every production failure becomes tomorrow's test case.



SESSION 11

Your plan for today

01

Set up and trace

Run the sample agent and explore its traces

02

Build a dataset

Turn a handful of traces into examples

03

Write three evaluators

One code check, one judge, one pairwise

04

Go online

Add one online evaluator and one automation rule

Course: academy.langchain.com/courses/building-reliable-agents